

Combinatorics and Cryptography

Lecture on Digital Signatures

Topics: EUF-CMA · Schnorr Signatures · Identification Protocols · Fiat-Shamir Transform

1. What Is a Digital Signature?

A **digital signature** is a cryptographic mechanism that allows a party, the *signer*, to attach an unforgeable mark of authenticity to a piece of data. Intuitively, it is the digital analogue of a handwritten signature on a paper document: anyone who holds the signer's *public key* can verify the mark, yet only the holder of the corresponding *secret key* can produce it.

Three fundamental properties are expected of any digital signature scheme.

1. **Correctness.** An honestly generated signature on any message always verifies successfully.
2. **Unforgeability.** An adversary without the secret key cannot produce a valid signature on any message, even after observing many signatures of its choice.
3. **Non-repudiation.** A signer cannot later deny having produced a valid signature, since doing so requires the secret key.

Formally, a digital signature scheme is a triple of efficient algorithms (KeyGen, Sign, Verify).

Definition 1.1: Digital Signature Scheme

A **digital signature scheme** is a triple of efficient algorithms such that (KeyGen, Sign, Verify):

- $\text{KeyGen}(\beta) \rightarrow (sk, pk)$ is a PPT algorithm that, given the security parameter, outputs a secret signing key sk and a public verification key pk .
- $\text{Sign}(sk, m) \rightarrow \sigma$ is a PPT algorithm that, given the secret key and a message $m \in \{0, 1\}^*$, outputs a signature σ .
- $\text{Verify}(pk, m, \sigma) \rightarrow \{0, 1\}$ is a deterministic algorithm that, given the public key, a message, and a signature, outputs 1 (accept) or 0 (reject).

Correctness requires that for all $(sk, pk) \xleftarrow{\$} \text{KeyGen}(\beta)$ and all messages m ,

$$\text{Verify}(pk, m, \text{Sign}(sk, m)) = 1.$$

2. Security Definition: EUF-CMA

The standard security notion for digital signatures is **existential unforgeability under chosen-message attack** (EUF-CMA). Before stating it formally, we introduce the key tool used to model the adversary's power.

Definition 2.1: Oracle and Oracle Access

An **oracle** for a function f is a black box that, on any input x , returns $f(x)$ in one step, without revealing anything about the internal workings of f . An algorithm with **oracle access** to f , written $\mathcal{A}^f$, may query the oracle on inputs of its choice, any polynomial number of times, and observes only the corresponding outputs.

In the context of signature schemes, the **signing oracle** $\mathcal{O}_{\text{Sign}}$ takes any message m as input and returns $\text{Sign}(sk, m)$, where sk is held by the challenger and never revealed. Giving the adversary access to this oracle models a realistic threat: an attacker that can observe (or even influence) what the legitimate signer signs in practice, without ever obtaining the secret key directly.

EUF-CMA then formalises the requirement that even an adversary with this power cannot produce a valid signature on any *new* message.

Definition 2.2: EUF-CMA Security

Let $\Sigma = (\text{KeyGen}, \text{Sign}, \text{Verify})$ be a signature scheme. Consider the following game between a challenger $\mathcal{C}$ and a PPT adversary $\mathcal{A}$:

1. **Setup.** $\mathcal{C}$ runs $(sk, pk) \xleftarrow{\$} \text{KeyGen}(\beta)$ and sends pk to $\mathcal{A}$.
2. **Query phase.** $\mathcal{A}^{\mathcal{O}_{\text{Sign}}}$ may query the signing oracle on any polynomial number of messages $m_1, \dots, m_q$ of its choice, obtaining signatures $\sigma_i = \text{Sign}(sk, m_i)$.
3. **Forgery.** $\mathcal{A}$ outputs a pair (m^*, σ^*) .

The adversary *wins* if $\text{Verify}(pk, m^*, \sigma^*) = 1$ and $m^* \notin \{m_1, \dots, m_q\}$. The scheme Σ is **EUFCMA secure** if for all PPT $\mathcal{A}$,

$$\text{Adv}_{\Sigma}^{\text{euf-cma}}(\mathcal{A}) := \Pr[\mathcal{A} \text{ wins}] = \text{negl}(\beta).$$

3. A First Attempt: RSA Signatures

Armed with the EUF-CMA definition, a natural first attempt at building a signature scheme is to reach for RSA. It provides a trapdoor: signing is computing an e -th root modulo n , which only the holder of the secret key d can do, while anyone can verify by raising to e . Let us see why the most direct construction fails, and how to fix it.

3.1 Textbook RSA Signatures**Construction 3.1: Textbook RSA Signature**

KeyGen(β): Generate an RSA modulus $n = pq$ and exponents e, d with $ed \equiv 1 \pmod{\phi(n)}$. Output $sk = d, pk = (n, e)$.

Sign(sk, m): Output $\sigma := m^d \pmod{n}$.

Verify(pk, m, σ): Accept iff $\sigma^e \equiv m \pmod{n}$.

Correctness. $(m^d)^e = m^{de} = m \pmod{n}$. $\square$

Correctness holds, but EUF-CMA security fails catastrophically. There are at least two independent attacks.

Attack 1: existential forgery without any signing query. The adversary picks an arbitrary $\sigma^* \xleftarrow{\$} \mathbb{Z}_n^*$, computes $m^* := (\sigma^*)^e \bmod n$, and outputs (m^*, σ^*) . By construction, $(\sigma^*)^e \equiv m^* \pmod{n}$, so the pair verifies. This is a valid forgery, obtained with *zero* interaction with the signing oracle. The scheme is broken before the adversary even asks a single question.

Attack 2: homomorphic forgery using the signing oracle. RSA has a multiplicative structure:

$$(m_1 m_2)^d = m_1^d \cdot m_2^d \pmod{n}.$$

An adversary that obtains signatures σ_1 on m_1 and σ_2 on m_2 immediately gets a valid signature $\sigma_1 \cdot \sigma_2 \bmod n$ on $m_1 \cdot m_2$, a message it never queried. Even a single signing query is enough to bootstrap new forgeries.

Both attacks stem from the same root cause: the message m enters the signing function *directly*, with no preprocessing, so the algebraic structure of RSA is fully exposed to the adversary.

3.2 RSA-FDH: Hashing to the Rescue

The fix is to interpose a hash function $H: \{0,1\}^* \rightarrow \mathbb{Z}_n$ between the message and the RSA operation. The resulting scheme is called **RSA Full Domain Hash** (RSA-FDH).

Construction 3.2: RSA-FDH Signature

KeyGen(β): same as textbook RSA — output $sk = d$, $pk = (n, e)$.

Sign(sk, m): Output $\sigma := H(m)^d \bmod n$.

Verify(pk, m, σ): Accept iff $\sigma^e \equiv H(m) \pmod{n}$.

The single change, replacing m with $H(m)$, is motivated by two independent reasons that happen to reinforce each other.

Efficiency. RSA operates on elements of $\mathbb{Z}_n$, which have a fixed size (typically 2048 or 4096 bits). A message m can be arbitrarily long. Applying H first compresses m to a fixed-size output that fits the domain of the RSA function, regardless of the length of m . Without this step, signing long messages would require either restricting message lengths or partitioning and signing each block separately, both unsatisfactory.

Security. Beyond efficiency, H must do real work to prevent the attacks that broke textbook RSA. Specifically, the three standard properties of a cryptographic hash function each neutralise one of the failure modes seen above.

Preimage resistance kills Attack 1. The forgery attack produced $m^* = (\sigma^*)^e \bmod n$ from a random σ^* . With FDH the adversary would need m^* such that $H(m^*) = (\sigma^*)^e \bmod n$, i.e. it would need to *invert* H on a given output. Preimage resistance says this is infeasible.

Collision resistance kills Attack 2. The homomorphic attack produced a signature on $m_1 m_2$ from signatures on m_1 and m_2 . With FDH the computation would require $H(m_1) \cdot H(m_2) \equiv H(m_1 m_2) \pmod{n}$, i.e. H would need to be multiplicatively homomorphic, the very structure we are trying to destroy. A collision-resistant hash function has no exploitable algebraic structure: its outputs look independent and unrelated across different inputs.

Second-preimage resistance prevents signature reuse. Even if an adversary holds a valid signature σ on m , it cannot find a different message $m' \neq m$ for which the same σ verifies, because that would require $H(m') = H(m)$, breaking second-preimage resistance.

Remark 3.1: FDH, PSS, and Real-World Standards

RSA-FDH can be proved EUF-CMA secure under the RSA assumption, and it is the cleanest theoretical construction in the hash-then-sign family: the proof is tight and the structure is transparent. For this reason it is the standard reference example in the literature.

In practice, however, FDH is not deployed. The obstacle is that “full domain hash” requires H to output elements of $\mathbb{Z}_n$, i.e. strings as long as the RSA modulus (2048–4096 bits). Standard hash functions (SHA-256, SHA-3) produce 256–512 bits and cannot be used directly.

The real-world standard is **RSA-PSS** (Probabilistic Signature Scheme, Bellare–Rogaway 1996), mandated by PKCS#1 v2.1 and FIPS 186-4. PSS introduces an explicit random *salt* before hashing, uses a mask generation function (MGF) to stretch outputs to the right size, and assembles a structured encoding block before applying RSA. The result also has a tight security proof reducible to RSA in the random oracle model, but works with standard-length hash functions. Many legacy systems instead use **PKCS#1 v1.5 padding**, which predates PSS and has no tight security proof. It has been the source of several serious attacks in practice, most notably the Bleichenbacher family of padding oracle attacks.

3.3 A Different Paradigm

RSA-FDH shows one route to a secure signature scheme: take a trapdoor permutation, hash the message into its domain, and sign the hash. The approach works, but it is somewhat monolithic: the security proof is tightly coupled to the specific algebraic structure of RSA, and the construction does not generalise easily to other settings.

There is a second, more modular paradigm that will occupy the rest of these notes. Rather than starting from a trapdoor function, one starts from an *interactive identification protocol* and makes it non-interactive via a hash function. This approach, namely the **Fiat–Shamir transform**, produces signature schemes whose security proofs share a common skeleton regardless of the underlying algebraic assumption, and it scales cleanly from discrete-log settings to RSA (as we will see with Guillou–Quisquater) to modern lattice-based constructions.

The first and most important example of this paradigm is the Schnorr signature scheme.

4. The Schnorr Signature Scheme

We now introduce a concrete and historically important signature scheme due to Claus-Peter Schnorr. It is built upon the discrete-logarithm assumption in a cyclic group, and its elegance lies in its simplicity.

4.1 Setup and Parameters

Let $\mathbb{G}$ be a cyclic group of prime order q , written multiplicatively, with generator g . All arithmetic in exponents is modulo q . We fix a cryptographic hash function¹ $H: \{0,1\}^* \rightarrow \mathbb{Z}_q$.

¹Note that q , i.e. the order of the group, is typically 256 bits for elliptic-curve instantiations, which matches exactly the output size of standard hash functions such as SHA-256. This stands in contrast to RSA-FDH, where the hash output must be as large as the RSA modulus (2048–4096 bits), ruling out standard hash functions. The Sigma+FS paradigm avoids this mismatch by design.

4.2 The Scheme

Construction 4.1: Schnorr Signatures

KeyGen(β):

- Sample $x \xleftarrow{\$} \mathbb{Z}_q$ and set $h := g^x \in \mathbb{G}$.
- Output $sk := x$ and $pk := (g, h)$.

Sign(sk, m):

1. Sample $r \xleftarrow{\$} \mathbb{Z}_q$ and compute the *commitment* $R := g^r$.
2. Compute the *challenge* $c := H(R \| m) \in \mathbb{Z}_q$.
3. Compute the *response* $s := r + c \cdot x \pmod{q}$.
4. Output $\sigma := (R, s)$.

Verify(pk, m, σ):

- Parse $\sigma = (R, s)$. Accept if and only if

$$g^s = R \cdot h^c, \quad \text{where } c := H(R \| m).$$

4.3 Correctness

For any honestly generated signature,

$$g^s = g^{r+cx} = g^r \cdot (g^x)^c = R \cdot h^c. \quad \square$$

5. Security Analysis

We argue *informally* that Schnorr is EUF-CMA secure under the discrete-logarithm assumption. The argument will not spell out all technicalities of a full reduction, but it identifies the precise algebraic reason why forging a signature is as hard as computing a discrete logarithm.

5.1 Key Structural Lemma

Lemma 5.1: Secret Recovery from Two Transcripts

Suppose there exist two valid signatures on the same message m and the same commitment R , but with *distinct* challenges:

$$(R, s) \text{ for challenge } c, \quad (R, s') \text{ for challenge } c', \quad c \neq c'.$$

Then one can efficiently compute the secret key x .

Proof. Both pairs satisfy the verification equation:

$$g^s = R \cdot h^c, \quad g^{s'} = R \cdot h^{c'}.$$

Dividing the first by the second:

$$g^{s-s'} = h^{c-c'} = g^{x(c-c')}.$$

Since q is prime and $c \neq c'$, the difference $(c - c')$ is invertible in $\mathbb{Z}_q$. Dividing both exponents by $(c - c')$:

$$x = (s - s') \cdot (c - c')^{-1} \in \mathbb{Z}_q. \quad \square$$

■

5.2 Why Forgery Implies Breaking Discrete Log

The lemma gives an immediate intuition for why Schnorr is secure. Consider any forger $\mathcal{A}$ that, without knowing x , produces a valid signature (R^*, s^*) on a fresh message m^* .

A valid signature must satisfy

$$g^{s^*} = R^* \cdot h^{c^*}, \quad c^* = H(R^* \| m^*).$$

The forger chose R^* *before* seeing the challenge c^* : the challenge is determined by R^* and m^* through H , so the forger is in the position of having committed to R^* first and only then having to respond to c^* .

Now ask: what would happen if we presented the *same* forger with the *same* commitment R^* but a *different* challenge $c' \neq c^*$? A truly capable forger, i.e. one that does not merely get lucky but genuinely “understands” how to forge on m^* , would produce a second valid response s' .² But Lemma 5.1 then immediately gives

$$x = (s^* - s') \cdot (c^* - c')^{-1},$$

recovering the secret key and solving the discrete-logarithm problem in $\mathbb{G}$.

Since discrete logarithm is assumed to be hard, no efficient forger can exist. The key insight is worth stating clearly:

Remark 5.1: The Core Intuition

A valid forgery implicitly encodes the ability to respond correctly to *any* challenge on a self-chosen commitment R^* . But that is exactly what it means to know the discrete logarithm x of h : a legitimate signer can respond to any challenge precisely because it knows x and can compute $s = r + cx$ for whatever c it receives. An impostor that lacks x is stuck: it must commit to R^* first, and then it can only hope the resulting challenge is the one specific value it has prepared for.

6. From Identification to Signatures

So far we have seen Schnorr signatures and argued informally that they are secure. A natural question is: where does this construction *come from*? Is it a clever one-off trick, or part of a larger pattern? It turns out to be the latter. Schnorr is a special case of a completely general methodology that converts any *identification protocol* with the right structure into a provably secure signature scheme. Understanding this connection is the main goal of this section.

²Formally, this is established by running the forger twice on the same internal random coins but with a different hash-function response for the query $H(R^* \| m^*)$. This is the *rewinding* technique, whose precise analysis (the Forking Lemma [5]) is left to a more advanced treatment.

Remark 6.1: The Fiat–Shamir Paradigm

The Schnorr signature scheme is an instance of the following general recipe: (1) design an interactive identification protocol with suitable algebraic properties (**Sigma protocol**); (2) apply the **Fiat–Shamir transform** [2] to make it non-interactive and message-binding. Any protocol fitting this recipe yields an EUF-CMA secure signature scheme.

6.1 Identification Protocols

An **identification protocol** is an interactive protocol between a *prover* P (holding sk) and a *verifier* V (holding pk), in which P convinces V of its identity without revealing sk .

A particularly structured class of such protocols proceeds in exactly three moves:

1. **Commitment.** P samples fresh randomness r and sends a commitment R to V .
2. **Challenge.** V samples a random challenge $c \xleftarrow{\$} \mathcal{CH}$, where $\mathcal{CH}$ is the challenge space, and sends it to P .
3. **Response.** P computes a response s from (sk, r, c) and sends it to V , who accepts or rejects.

For the **Schnorr identification protocol** specifically, the prover shows their identity by proving knowledge of the private key x , proceeding as follows:

Construction 6.1: Schnorr Identification Protocol

Commitment. P samples $r \xleftarrow{\$} \mathbb{Z}_q$ and sends $R := g^r$.

Challenge. V sends $c \xleftarrow{\$} \mathbb{Z}_q$.

Response. P sends $s := r + cx \in \mathbb{Z}_q$.

Verification. V accepts iff $g^s = R \cdot h^c$.

6.2 The Verifier Learns Nothing About the Secret

A natural question is whether the interaction itself leaks information about x . After all, V sees a transcript (R, c, s) where $s = r + cx$; could it not recover x from this? The answer is no.

Note first that V always accepts an honest prover: if $s = r + cx$ and $R = g^r$, then $g^s = g^{r+cx} = g^r \cdot (g^x)^c = R \cdot h^c$, exactly as required. Now for the more interesting claim.

Theorem 6.1: The Interaction Reveals No Information About x

The distribution of a transcript (R, c, s) produced in an honest interaction between P (knowing x) and V is identical to the distribution of a transcript produced by a simulator that does *not* know x . In particular, V cannot learn anything about x from the interaction that it could not have computed on its own.

Proof. We describe the simulator Sim , and then argue the two distributions coincide.

The simulator. Given only the public key $h = g^x$ (and not x itself), Sim proceeds as follows:

1. Sample $s \xleftarrow{\$} \mathbb{Z}_q$ and $c \xleftarrow{\$} \mathbb{Z}_q$ uniformly at random.
2. Set $R := g^s \cdot h^{-c}$.
3. Output the transcript (R, c, s) .

The simulated transcript is valid. By construction,

$$R \cdot h^c = g^s \cdot h^{-c} \cdot h^c = g^s,$$

so $g^s = R \cdot h^c$: the verifier would accept this transcript.

The distributions are identical. In an honest execution, the prover samples $r \xleftarrow{\$} \mathbb{Z}_q$, sets $R = g^r$, receives c from the verifier, and responds with $s = r + cx$. Consider the joint distribution of (R, c, s) :

- c is chosen uniformly by the verifier: $c \sim \text{Unif}(\mathbb{Z}_q)$.
- r is chosen uniformly by the prover, independently of c and x : thus $s = r + cx \sim \text{Unif}(\mathbb{Z}_q)$, because adding the fixed quantity cx to a uniform variable r yields a uniform variable.
- $R = g^r$. Since r is uniform and $s = r + cx$, we have $r = s - cx$, so $R = g^{s-cx} = g^s \cdot h^{-c}$.

In summary: (R, c, s) is a uniformly random triple in $\mathbb{G} \times \mathbb{Z}_q \times \mathbb{Z}_q$ subject to the single constraint $R = g^s \cdot h^{-c}$.

The simulator produces exactly the same distribution: s and c are sampled uniformly, and R is set to $g^s \cdot h^{-c}$. The two joint distributions are therefore *identical*. Not merely computationally indistinguishable, but equal as probability distributions.

Since any information that V could extract from a real transcript it could equally extract from the simulated one (which was produced without x), the interaction reveals nothing about x beyond what is already implicit in the public key $h = g^x$. ■

Remark 6.2: Honest-Verifier Zero-Knowledge and Sigma Protocols

The property established in Theorem 6.1 is called **honest-verifier zero-knowledge** (HVZK): the existence of a simulator that produces transcripts identically distributed to real ones, *assuming the verifier sends a uniformly random challenge*. The qualifier “honest-verifier” is important: the argument breaks down if the verifier is allowed to choose challenges maliciously, because Sim picks c before R , whereas in the real protocol R comes first.

6.3 Special Soundness

We have already seen this property in action. In Section 4, the key step was: given two valid signatures (R, s, c) and (R, s', c') on the same message with the same commitment R but distinct challenges $c \neq c'$, one recovers $x = (s - s')(c - c')^{-1}$. We are now simply giving this property a name.

A three-move protocol is **specialy sound** if from any two accepting transcripts (R, c, s) and (R, c', s') sharing the same commitment but with $c \neq c'$, one can efficiently extract sk . Lemma 5.1 is precisely the statement that the Schnorr protocol is specially sound.

We now have all three properties on the table — completeness, special soundness, and honest-verifier zero-knowledge — and Schnorr satisfies all of them. This is not a coincidence: it is the definition of a Sigma protocol and Schnorr protocol is an example of Sigma protocol.

Definition 6.1: Sigma Protocol (Σ -protocol)

A three-move identification protocol is a **Sigma protocol** if it satisfies *completeness*, *special soundness*, and *honest-verifier zero-knowledge*.

Remark 6.3: References

The theory of Sigma protocols is developed in detail in [6] (freely available lecture notes). The Fiat–Shamir transform, described next, turns any Sigma protocol into an EUF-CMA secure signature scheme, making the concept central not only to identification, but to the design of practical signatures.

6.4 ★ A Second Example: Guillou–Quisquater

Before stating the Fiat–Shamir transform in full generality, it is worth seeing a second Sigma protocol built on a completely different mathematical foundation. The **Guillou–Quisquater (GQ) protocol** [1] does for RSA what Schnorr does for discrete logarithm.

Setup. Let $n = pq$ be an RSA modulus. Unlike standard RSA, this protocol requires e to be a *prime* exponent coprime to $\phi(n)$; primality is not an RSA requirement in general, but it is essential for the soundness argument below. The prover samples $u \xleftarrow{\$} \mathbb{Z}_n^*$, computes $v := u^e \bmod n$, and publishes v as its public key. It knows the e -th root of v because it constructed the pair in the easy direction: pick the root first, derive v from it. The hardness guarantee is that an adversary who sees only (n, e, v) cannot recover u , that is the RSA problem.

Construction 6.2: GQ Identification Protocol

Commitment. P samples $r \xleftarrow{\$} \mathbb{Z}_n^*$ and sends $R := r^e \bmod n$.

Challenge. V sends $c \xleftarrow{\$} \{0, \dots, e-1\}$.

Response. P sends $s := r \cdot u^c \bmod n$.

Verification. V accepts iff $s^e \equiv R \cdot v^c \pmod{n}$.

Correctness. $s^e = (r \cdot u^c)^e = r^e \cdot (u^e)^c = R \cdot v^c \pmod{n}$. □

Special soundness. Given two accepting transcripts (R, c, s) and (R, c', s') with $c \neq c'$:

$$s^e \equiv R \cdot v^c \pmod{n}, \quad s'^e \equiv R \cdot v^{c'} \pmod{n}.$$

Dividing: $(s/s')^e \equiv v^{c-c'} \pmod{n}$. Since e is prime and $0 < |c-c'| < e$, we have $\gcd(c-c', e) = 1$, so Bézout gives $\alpha, \beta \in \mathbb{Z}$ with $\alpha(c-c') + \beta e = 1$. Set $w := (s/s')^\alpha \cdot v^\beta \bmod n$. Then:

$$w^e = (s/s')^{e\alpha} \cdot v^{e\beta} \equiv v^{\alpha(c-c')} \cdot v^{e\beta} = v^{\alpha(c-c') + \beta e} = v^1 = v \pmod{n}.$$

Since $\gcd(e, \phi(n)) = 1$, the map $x \mapsto x^e$ is a bijection on $\mathbb{Z}_n^*$, so v has a unique e -th root. As u is that root by construction, we conclude

$$u \equiv w = (s/s')^\alpha \cdot v^\beta \pmod{n},$$

recovering the secret key. The structure is the same as in Lemma 5.1: two responses to the same commitment, different challenges, secret extracted. □

Honest-Verifier Zero-Knowledge. The argument is identical to the Schnorr case of Theorem 6.1. The simulator samples $s \xleftarrow{\$} \mathbb{Z}_n^*$ and $c \xleftarrow{\$} \{0, \dots, e-1\}$ uniformly, then sets $R := s^e \cdot v^{-c} \pmod{n}$. By construction, $s^e \equiv R \cdot v^c \pmod{n}$, so the transcript (R, c, s) is always accepting. As in the previous proof, in a real execution the prover's randomness $r \xleftarrow{\$} \mathbb{Z}_n^*$ is uniform, and for fixed c the map $r \mapsto s = r \cdot u^c$ is a permutation of $\mathbb{Z}_n^*$. Hence s is uniform, and

$$R = r^e = (s \cdot u^{-c})^e = s^e \cdot v^{-c} \pmod{n}.$$

Thus the simulator produces exactly the same distribution as an honest interaction. □

6.5 The Fiat–Shamir Transform

The key insight of Fiat and Shamir is that in a three-move identification protocol, the verifier’s challenge serves no purpose other than to prevent the prover from predicting it in advance. In the non-interactive setting one can replace it by the output of a hash function applied to the commitment (and, to bind the scheme to a specific message, to the message itself).

Definition 6.2: Fiat–Shamir Transform

Let $\Pi = (\text{KeyGen}, P, V)$ be a three-move identification protocol with challenge space $\mathcal{CH}$. Let $H: \{0, 1\}^* \rightarrow \mathcal{CH}$ be a hash function. The **Fiat–Shamir signature scheme** $\text{FS}[\Pi, H]$ is defined by:

- **KeyGen**: run the identification scheme’s key generation.
- **Sign**(sk, m): run P to get commitment R ; set $c := H(R \parallel m)$; compute response s from (sk, r, c) ; output $\sigma = (R, s)$.
- **Verify**(pk, m, σ): parse $\sigma = (R, s)$, recompute $c := H(R \parallel m)$, run $V(pk, R, c, s)$.

6.6 Security of the Fiat–Shamir Transform

The following is the main result of this section, and the reason Sigma protocols matter for the design of digital signatures. The argument follows the same structure as the informal security analysis of Section 4: a forger capable of responding to two distinct challenges on the same commitment R^* would allow extraction of sk by special soundness, contradicting the hardness assumption underlying Π . We omit the details.

Corollary 6.1: Any Sigma Protocol Gives a Secure Signature Scheme

Let Π be a Sigma protocol. Then $\text{FS}[\Pi, H]$, with H modelled as a random oracle, is an EUF-CMA secure digital signature scheme.

In other words: *designing a new Sigma protocol is the same as designing a new digital signature scheme*. This observation has generated an enormous variety of practical signature schemes, from classical discrete-log settings to modern lattice-based constructions.

7. ★ A Historical Detour: DSA

The **Digital Signature Algorithm** (DSA), standardised by NIST in 1994, operates in the same arithmetic setting as Schnorr — cyclic group $\mathbb{G}$ of prime order q , generator g , secret key $x \xleftarrow{\$} \mathbb{Z}_q$, public key $h = g^x$ — but with one decisive structural difference in how the challenge is formed.

Construction 7.1: DSA

KeyGen(β): same as Schnorr — output $sk = x$, $pk = (g, h)$ with $h = g^x$.

Sign(sk, m):

1. Sample $r \xleftarrow{\$} \mathbb{Z}_q^*$, compute $R := g^r$, set $r_1 := R \bmod q$.
2. Compute $c := H(m) \in \mathbb{Z}_q$. *(only m enters the hash; R does not)*
3. Compute $s := r^{-1}(c + x \cdot r_1) \bmod q$.
4. Output $\sigma := (r_1, s)$.

Verify(pk, m, σ): Parse $\sigma = (r_1, s)$, set $c := H(m)$, and accept iff

$$(g^{c/s} \cdot h^{r_1/s}) \bmod q = r_1.$$

Correctness. If honest: $s = r^{-1}(c + xr_1)$, so $(c + xr_1)/s = r$, and $g^{c/s} \cdot h^{r_1/s} = g^{(c+xr_1)/s} = g^r = R$, whose residue mod q is r_1 . $\square$

Why DSA Is Not a Sigma Protocol

The single difference — $c = H(m)$ rather than $c = H(R \parallel m)$ — has far-reaching consequences. Recall that in the Schnorr HVZK proof (Theorem 6.1), the simulator was free to choose s and c at random and then set $R = g^s \cdot h^{-c}$, making the fake transcript indistinguishable from a real one. This worked because c depended on R only through the hash, which the simulator could coordinate freely.

In DSA, $c = H(m)$ is fixed by the message alone, independently of R . A simulator has no way to coordinate R and c : choosing s and r_1 freely still leaves the constraint $g^{c/s} \cdot h^{r_1/s} \bmod q = r_1$ unsatisfied without knowing x . The HVZK property fails, and DSA is therefore *not* a Sigma protocol and cannot be obtained via Fiat–Shamir.

An Informal Security Argument

A forger must produce (r_1^*, s^*) for a fresh message m^* satisfying $g^{c^*/s^*} \cdot h^{r_1^*/s^*} \bmod q = r_1^*$, where $c^* = H(m^*)$. Letting $k = (c^* + xr_1^*)/s^*$, this says $g^k \bmod q = r_1^*$: the forger must find an exponent k whose group element reduces to a specific value mod q . Since r_1^* appears on both sides and x is unknown, the forger faces a circular constraint: any choice of s^* determines k through x , which it does not have. Guessing k directly is as hard as computing a discrete logarithm.

Caveat 7.1: No Tight Security Proof Is Known for DSA

The informal argument above identifies the right circularity but falls well short of a proof. In Schnorr, two forgeries on the same R immediately yield x by Lemma 5.1 — a clean algebraic extraction. For DSA no such clean extraction is known: there is no standard-model tight reduction from discrete log or any other well-studied assumption.

The best known security results require either the **generic group model** (a strong idealisation) or non-standard assumptions. This theoretical gap, together with a history of catastrophic failures when r is reused or biased, is why the community has migrated towards **ECDSA** (DSA on elliptic curves, the standard in TLS, SSH, and Bitcoin) and more recently **EdDSA** (Ed25519), which returns to the Schnorr framework and enjoys a clean tight proof.

References

- [1] L. C. Guillou and J.-J. Quisquater. *A Practical Zero-Knowledge Protocol Fitted to Security Microprocessors*. EUROCRYPT 1988.
- [2] A. Fiat and A. Shamir. *How to Prove Yourself: Practical Solutions to Identification and Signature Problems*. CRYPTO 1986.
- [3] C.-P. Schnorr. *Efficient Signature Generation by Smart Cards*. Journal of Cryptology, 4(3), 1991.

- [4] M. Bellare and P. Rogaway. *Random Oracles are Practical: A Paradigm for Designing Efficient Protocols*. CCS 1993.
- [5] D. Pointcheval and J. Stern. *Security Arguments for Digital Signatures and Blind Signatures*. Journal of Cryptology, 13(3), 2000.
- [6] I. Damgård. *On Σ -protocols*. Lecture Notes, Aarhus University, 2010. <https://cs.au.dk/~ivan/Sigma.pdf>